James Kendrick

(210) 429-6569 | kendrickj5@yahoo.com | GitHub | LinkedIn | Portfolio

EDUCATION

University of Illinois Urbana-Champaign Grad Date: May 2027

B.S. Mathematics and Computer Science GPA**:** 3.86/4.00

EXPERIENCE

Fab2 | Software Engineer Intern | San Francisco, CA (Incoming) Sep – Dec 2026

- Developing data-centric applications and infrastructure for a software-defined semiconductor fab.

Meta | Data Engineer Intern | Seattle, WA Jun – Aug 2026

- Architected an artifact-agnostic drift detection framework that keeps AI-generated artifacts in sync with the specs they derive from; used a scalable design where each artifact registers its logic behind a shared core that orchestrates execution, reporting, and logging.
- Implemented adapters for two artifact classes covering 36 items and 12 specs, unmarshalling spec text, Thrift types, and iOS/Android source code into canonical, diffable models; surfaced 20+ actionable drift items on first run.
- Built a Python library and agent skill letting AI tools auto-remediate drift, backed by a new server-side endpoint written in PHP that dropped legacy dev server dependencies, which cut a single edit from 5+ minutes and a manual 2FA confirmation to ~10 seconds.
- Shipped the messaging observability data layer for Meta's standalone Seller Marketplace app: owned pipelines computing health metrics for 4 user-facing flows, plus DQ validation, alerting wiring, and a leadership-facing dashboard.

Meta | Data Engineer Intern | New York City, NY May – Aug 2025

- Designed a configuration-driven Python ETL framework that horizontally scaled dimensional modeling across the Instagram Graph domain, enabling Data Scientists to define arbitrary event chains and deploy multi-terabyte pipelines in minutes.
- Optimized the framework's procedural SQL translator to execute a disk-backed, Breadth-First Search (BFS) data cube lattice, bypassing Presto out-of-memory limitations and slashing peak memory utilization by ~80% across all generated pipelines.
- Improved private follow surface attribution accuracy by 4% in existing pipelines and overall surface attribution by 20% in new ones, impacting 30+ downstream metrics consumed by 100+ users and enabling more reliable product decision-making.

PROJECTS | View All

Grimoire | Website | GitHub | Demo Apr 2026 – Present

- Designed a meta-runtime for orchestrating polyglot computation in Go, enabling uninstrumented, language-agnostic functions to be run via generated CLI commands, YAML configs, and a custom DSL; built around a pluggable runtime-adapter interface (Python and Go shipped, extensible to any language) to enable a zero-boilerplate plugin system.
- Implemented AST-based signature extraction with tree-sitter to derive typed argument schemas directly from source.
- Built an abstraction that auto-provisions and caches execution environments keyed by the SHA-256 hash of dependency and source file manifests for freshness tracking – cutting warm-invocation overhead from ~4+ seconds to ~50ms (>80x speedup).

Sprite | Website | GitHub | Demo Jan 2025 – Present

- Developed a high-performance C++ CLI tool for Zsh that augments directory navigation, utilizing an embedded SQLite engine to achieve sub-hundred-millisecond latency for path resolution and command execution.
- Engineered a context-aware ranking algorithm that optimizes autocompletion suggestions based on historical usage patterns.
- Designed a "quick-nav" execution engine that wraps standard shell binaries, enabling users to execute commands on fuzzy-matched paths instantly, reducing keystrokes by ~40% during deep directory traversal.

Court Vision | Website | GitHub | Demo Jan 2024 – Present

- Built a modular Python/FastAPI ETL data platform that ingests daily NBA stats and live box scores from multiple external APIs, writing to PostgreSQL with idempotent upserts, retry/circuit-breaker resilience, and per-pipeline audit trails.
- Engineered a Genetic Algorithm in Go to solve an NP-Hard Lineup Scheduling problem, utilizing concurrent evolution to optimize streaming strategies across ~10^9 combinations, outperforming greedy approaches by ~15% in projected fantasy points.
- Embedded **SQLMate** (another personal project of mine; Website | GitHub | Demo) as a visual, no-code query interface for the public REST API, enabling non-technical users to explore the structured fantasy basketball dataset without writing SQL.

TECHNICAL SKILLS

Programming Languages: Python, Go, SQL, C++, JavaScript/TypeScript

Infrastructure and Tools: Docker, Git, Linux, AWS/GCP/Azure, CI/CD, Postgres, Agentic Coding Tools (e.g., Claude Code, Cursor)